

CAREER ROADMAP · AI ENGINEER · 2026 EDITION

AI Engineer in 6 Months

The Complete Zero-to-Job-Ready Roadmap

180-Day Plan · 15 Portfolio Projects · 200+ Interview Q&As · Resume + LinkedIn Kit

CRAFTED BY @dipanshu.decoded

8

MODULES

6

MONTHS

15+

PROJECTS

200+

INTERVIEW Q&As

WHAT'S INSIDE

- Zero-to-job-ready curriculum across 6 months / 24 weeks
- A day-by-day 180-day study plan with checkpoints every 30 days
- 15 premium, resume-ready portfolio projects with folder structures
- 200+ curated interview questions across Python, ML, DL, GenAI, RAG & Agents
- AI system design case studies, resume templates & a 30-day job-search plan

Table of Contents

Part 1 — The AI Engineer Career Blueprint	03
Part 2 — The 6-Month Curriculum (Months 1–6)	05
Part 3 — The 180-Day / 24-Week Study Plan	13
Part 4 — 15 Premium Portfolio Projects	19
Part 5 — Coding & Interview Question Banks	23
Part 6 — AI System Design	27
Part 7 — GitHub & Portfolio Strategy	29
Part 8 — The AI Engineer Resume	30
Part 9 — LinkedIn & Personal Branding	31
Part 10 — The 30-Day Job Search Plan	32
Part 11 — Interview Cheat Sheets	33
Part 12 — Final Job-Ready Checklist & Your Journey	34

HOW TO USE THIS BOOK

Read Parts 1–3 in order — they are the spine of your 180 days. Parts 4–12 are reference sections: dip into projects, question banks and interview sheets as you reach the matching week. Not another "watch tutorials and hope" plan — this is the exact zero-to-job-ready sequence, python foundations to a hired, interview-ready AI engineer in six focused months, with a milestone check at every stage.

WHO THIS ROADMAP IS FOR	WHAT IT TAKES
Complete beginners — comfortable with basic programming logic, ready to trade 6 months of casual learning for 6 months of focused, sequenced building.	Build every week. Watching content without shipping code doesn't count as progress.
Career switchers — coming from software, data, or a non-tech field, who need a credible portfolio and interview reps, not just more theory.	Don't skip the milestone check — it's the only honest signal that a month actually landed.

The AI Engineer Career Blueprint

Before you write a single line of code, understand the job you're training for. This part maps the whole territory — the terms, the roles, the stack — so every week that follows has a place to slot into.

What is Artificial Intelligence?

Artificial Intelligence is the broad field of building systems that perform tasks which normally require human intelligence — understanding language, recognizing images, making decisions. **Machine Learning** is the dominant approach to AI today: instead of hand-coding rules, you show a system examples and let it learn the pattern. **Deep Learning** is a subset of ML that uses layered neural networks to learn from large, messy, unstructured data (images, audio, text). **Generative AI** is a subset of deep learning — models (mostly transformers) trained to generate new content: text, code, images. Every layer sits inside the one before it.

Artificial Intelligence	Machine Learning	Deep Learning	Generative AI
The whole field	Learn patterns from data	Neural nets, unstructured data	Transformers, LLMs

What does an AI Engineer actually do?

An AI Engineer builds **products powered by models** — not the models themselves from scratch. Day to day: calling and orchestrating LLM APIs, building retrieval pipelines over company data, writing backend services that serve predictions, evaluating and monitoring model output in production, and wiring agents/tools together. Roughly 70% of the job is software engineering; 30% is model-specific reasoning (prompting, evaluation, fine-tuning when needed).

PRO TIP

Hiring managers in 2026 care far more about "can you ship a working AI feature end-to-end" than "can you derive backpropagation on a whiteboard." Depth in math helps you debug; depth in shipping gets you hired.

AI Engineer vs ML Engineer vs Data Scientist

ROLE	CORE FOCUS	TYPICAL OUTPUT
Data Scientist	Analysis, experimentation, statistics	Insights, dashboards, A/B tests
ML Engineer	Training, deploying and maintaining custom models	Trained models in production, pipelines
AI Engineer	Building applications on top of foundation models (LLMs, embeddings)	AI features, agents, RAG systems, APIs

Skills required in 2026

- Python + clean software engineering

- SQL and data manipulation
- Core ML & deep learning fundamentals
- PyTorch for model work
- Transformers & LLM internals
- Prompt engineering & structured outputs
- RAG & vector databases
- AI agents & tool calling
- FastAPI, Docker, cloud basics
- Evaluation, safety & observability

The typical AI tech stack (2026)

LAYER	TOOLS
Language & core	Python, Git, SQL
ML / DL	NumPy, Pandas, scikit-learn, PyTorch
GenAI / LLM	OpenAI / Anthropic / open-weight APIs, LangChain, LlamaIndex, Hugging Face
Retrieval	FAISS, Chroma, Pinecone
Serving & infra	FastAPI, Docker, AWS/Azure/GCP basics
MLOps	MLflow, CI/CD, monitoring/observability tools

COMMON MISTAKE

Beginners try to learn every framework in parallel. Go deep on one LLM API, one vector DB and one orchestration framework first — depth on one stack ships projects; breadth across ten stalls you at tutorials.

Entry-level roles to target

AI Engineer · Junior ML Engineer · Generative AI Engineer · Applied AI Engineer · AI/ML Developer · Machine Learning Intern → full-time convert. In India, look at product companies, GenAI-focused startups, and services companies' new AI practices — all three hire freshers with strong projects.

What recruiters actually expect from freshers

- 2–3 non-tutorial projects with a live demo and a clean GitHub
- Ability to explain why a model/approach was chosen, not just that it was used
- Comfort with APIs, Git and basic deployment — not just notebooks
- Clear communication about trade-offs (cost, latency, accuracy)

The AI Engineer Skill Tree

Foundations	Classical ML	Deep Learning	Generative AI	Production
Python	Regression	Neural nets	Transformers	FastAPI
Git/GitHub	Classification	PyTorch	LLM APIs	Docker
NumPy/Pandas	Clustering	CNNs	RAG	MLOps
Math intuition	Feature eng.	NLP basics	Agents	Interviews

The 6-Month Curriculum

Six months, six layers. Each month below lists exactly what to learn and why it matters, closing with one flagship project that proves you can use it. The day-by-day schedule that turns this into 180 daily sessions is in Part 3.

MONTH	FOCUS	CLOSING PROJECT
1	Python, Math & Data Foundations	Exploratory Data Analysis project
2	Machine Learning	End-to-end production-style ML project
3	Deep Learning (CV + NLP)	Image classifier + text classifier
4	Generative AI & LLM Engineering	Chat-with-your-PDFs assistant (RAG)
5	Advanced GenAI & AI Agents	Production RAG or AI agent
6	MLOps, Deployment & Job Prep	Deployed, monitored AI service + job search

MONTH 1 · FOUNDATIONS

Python, Math & Data Foundations

Skipping fundamentals to jump straight into LLMs is the single most common reason people stall out around month three. ML foundations aren't optional groundwork — they're what let you debug a model instead of just calling `.fit()` and hoping.

Week 1 — Python Fundamentals

Variables, data types, operators, conditions, loops, functions, strings, lists, tuples, sets, dictionaries, I/O, exception handling. Practice every concept with small scripts before moving on — fluency here is what makes every later week fast.

```
# exception handling – the pattern you'll use constantly
try:
    result = 10 / user_input
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Enter a valid number")
```

Exercise: build a command-line grade calculator handling invalid input. **Interview Q:** difference between a list and a tuple? (mutability, use case, memory).

Week 2 — Intermediate Python

OOP (classes, objects, inheritance, polymorphism, encapsulation), modules & packages, file handling, JSON, consuming APIs, virtual environments, pip, Git and GitHub basics.

MINI PROJECT

Build a Python app that consumes a public API (e.g. weather or currency-exchange) — parse JSON, handle errors, and print a clean report. Push it to GitHub with a README.

Week 3 — NumPy + Pandas

NumPy arrays, indexing, broadcasting; Pandas Series and DataFrames, cleaning, missing values, filtering, groupby, merge, transformation. **Exercise:** clean and analyze a real messy CSV dataset end to end.

Week 4 — Mathematics for AI

Linear algebra (vectors, matrices, dot products, matrix multiplication) explains how data and model weights are represented. Probability & statistics (mean, median, variance, distributions) explain uncertainty and evaluation. Calculus (derivatives, gradients, gradient descent) explains *how models learn* — a model improves by moving its weights slightly against the gradient of its error.

REMEMBER THIS

You don't need to derive proofs. You need the intuition: gradient descent = "take small steps downhill on the error surface." That sentence carries you through every ML and DL interview question about optimization.

MILESTONE CHECK

Full exploratory data analysis (EDA) on a real dataset: load with Pandas, clean missing/duplicate data, engineer 2–3 new features, visualize distributions and correlations, and summarize 5 findings in a short report.

COMMON TRAP

Watching hours of "ML in Python" videos without writing a single model from scratch — recognition isn't the same as recall.

Machine Learning

Week 5 — ML Fundamentals

Supervised vs. unsupervised vs. reinforcement learning; features and labels; train/validation/test splits; data leakage; overfitting vs. underfitting; the bias–variance trade-off.

Week 6 — Regression

Linear, multiple and polynomial regression; Ridge and Lasso regularization; evaluation with MAE, MSE, RMSE, R^2 .

PROJECT

House Price Predictor — regression with regularization and proper cross-validation.

Week 7 — Classification

Logistic regression, KNN, decision trees, random forests, SVM, Naive Bayes, gradient boosting, XGBoost. Metrics: accuracy, precision, recall, F1, confusion matrix, ROC-AUC — and when accuracy alone lies to you (imbalanced classes).

PROJECT

Customer Churn Predictor — classification with a full metrics report, not just accuracy.

Week 8 — Unsupervised Learning & Feature Engineering

K-Means, hierarchical clustering, PCA; feature selection/extraction, encoding, scaling, normalization; handling imbalanced data; hyperparameter tuning with GridSearchCV/RandomizedSearchCV; cross-validation.

INTERVIEW TIP

"How would you handle an imbalanced dataset?" is asked constantly. Have three answers ready: resampling (SMOTE/undersampling), class-weighted loss, and choosing the right metric (F1/PR-AUC over accuracy).

MILESTONE CHECK

An end-to-end production-style ML pipeline: data ingestion → cleaning → feature engineering → model training with cross-validation → hyperparameter tuning → a saved, reloadable model artifact.

Deep Learning

Week 9 — Neural Networks

Perceptrons, neurons, layers, weights, biases, activation functions, loss functions, forward propagation, backpropagation, optimizers, learning rate, epochs, batch size — the vocabulary every DL framework is built on.

Week 10 — PyTorch

Tensors, Datasets and DataLoaders, model definitions, training loops, loss functions, optimizers, GPU training, saving/loading models. PyTorch is this roadmap's primary deep-learning framework — go deep here rather than splitting time with TensorFlow.

```
for epoch in range(epochs):
    for x, y in loader:
        optimizer.zero_grad()
        pred = model(x)
        loss = loss_fn(pred, y)
        loss.backward()
        optimizer.step()
```

Line by line: clear old gradients → forward pass → compute loss → backprop the gradients → update weights. Every PyTorch training loop you'll ever write is a variation of this.

Week 11 — Computer Vision

CNNs, convolution, pooling, image preprocessing, data augmentation, transfer learning, image classification.

PROJECT

Image classifier using transfer learning on a pretrained CNN, deployed behind a simple demo UI.

Week 12 — NLP

Tokenization, stop words, stemming, lemmatization, Bag of Words, TF-IDF, word embeddings, Word2Vec, attention, and a first look at transformers (deep dive in Month 4).

PROJECT

Sentiment/text classifier — TF-IDF or embedding features into a classical or small neural classifier.

COMMON TRAP

Training a CNN from scratch on a small dataset. In 2026, transfer learning is the default — fine-tune a pretrained backbone unless you have a genuinely large, unique dataset.

Generative AI & LLM Engineering

This is the core of what makes you hireable as an *AI Engineer* specifically. Slow down here.

Week 13 — Transformers

Self-attention lets every token look at every other token and weigh how relevant each is; multi-head attention runs several of these weightings in parallel to capture different relationships. Positional encoding injects word order (attention alone is order-blind). An encoder reads input into a representation; a decoder generates output from it — GPT-style models are decoder-only. Tokens are the model's atomic units of text; the context window is how many tokens it can "see" at once; embeddings are the numeric vectors that represent meaning.

Week 14 — Large Language Models

Pretraining builds general language ability from huge text corpora; fine-tuning and instruction tuning specialize a model to follow instructions and behave helpfully. At inference, **temperature** controls randomness (0 = deterministic, higher = more creative) and **top-p** restricts sampling to the most probable tokens. Hallucinations are confident, fabricated answers — a core risk to design around, not an edge case. Prompt engineering, structured outputs (JSON schemas) and function/tool calling are how you make an LLM reliable inside a real application.

Week 15 — LLM APIs & Application Development

System vs. user messages, streaming responses, structured/JSON output, tool calling, error handling, rate limits, cost optimization (token budgets, caching, smaller models for simple steps), and security (prompt-injection awareness, never trusting model output as executable).

PROJECT

An AI assistant application: streaming chat, system-prompt persona, structured tool calls to a small backend function.

COMMON TRAP

Treating LLM APIs as static tools. Model names, pricing, context limits and tool-calling formats change fast — always check the provider's current docs before shipping, and design your integration layer so swapping models is a config change, not a rewrite.

Week 16 — Retrieval-Augmented Generation (RAG)

RAG grounds an LLM in your own data instead of relying on what it memorized. Pipeline: load documents → chunk them → embed chunks into vectors → store in a vector database → at query time, embed the question, retrieve the most similar chunks (optionally rerank them) → construct a context prompt → generate a grounded answer. Add metadata filtering (e.g. by document type/date) and hybrid search (keyword + vector) for production-grade retrieval, and always run RAG evaluation (faithfulness, relevance) before shipping.

TOOL	ROLE
LangChain / LlamaIndex	Orchestration — chaining loaders, retrievers and LLM calls
FAISS / Chroma	Local/open-source vector search
Pinecone	Managed, scalable vector database
Hugging Face	Open embedding & open-weight models

MILESTONE CHECK

"Chat With Your PDFs" — a full RAG assistant: upload PDFs, chunk and embed them, store in a vector DB, retrieve and answer questions with cited sources.

Advanced Generative AI & AI Agents

Week 17 — Advanced RAG (optional depth)

Chunking strategies (fixed vs. semantic), query rewriting, reranking models, hybrid retrieval, multi-query retrieval, context compression, RAG evaluation frameworks, and the architecture of production RAG systems (caching, async retrieval, fallback behavior).

Week 18 — AI Agents

An agent is an LLM given a loop: reason about a goal, choose a tool, observe the result, repeat until done. Core pieces: planning/reasoning, tool definitions, memory (short-term context, long-term stores), and — for anything consequential — a human-in-the-loop checkpoint before the agent acts. Multi-agent systems split work across specialized agents (a researcher, a writer, a reviewer). The biggest engineering challenge is reliability: agents fail silently in creative ways, so guardrails and evaluation matter more than clever prompting.

PROJECT

A research agent: takes a topic, searches/retrieves sources via tools, synthesizes a structured report.

Week 19 — Model Customization (optional / advanced)

Fine-tune only when prompting and RAG genuinely can't get you the behavior or style you need (RAG is almost always cheaper and faster to iterate on). LoRA and QLoRA fine-tune a small set of added parameters instead of the whole model — dramatically cheaper. PEFT is the Hugging Face library for these techniques; quantization shrinks a model's memory footprint for cheaper inference.

Week 20 — AI Evaluation & Safety

Evaluate LLM and RAG outputs on groundedness (is it supported by the retrieved context?), relevance, and hallucination rate. Test for prompt injection and jailbreaks. Add guardrails (input/output filtering), respect data privacy, and instrument observability (logging prompts, responses, latency, cost) so you can debug production behavior.

INTERVIEW TIP

"How do you reduce hallucinations?" — strong answer: ground with RAG, cite sources, constrain output format, add a self-check/verification step, and evaluate with a held-out test set before shipping.

MILESTONE CHECK

Take your Month 4 RAG project (or a new one) to production quality: add reranking, evaluation metrics, and either an agent loop or multi-step reasoning on top.

MLOps, Deployment & Job Preparation

Week 21 — Backend for AI Engineers

FastAPI for REST APIs, Pydantic for request/response validation, async basics for concurrent request handling, model serving patterns, basic authentication, logging and error handling.

PROJECT

Expose one of your ML/AI models behind a FastAPI endpoint with input validation and error handling.

Week 22 — Docker & Cloud

Dockerfiles, images vs. containers, Docker Compose for multi-service apps, environment variables and secrets management, and a working overview of AWS/Azure/GCP and GPU deployment options. Deploy your AI application to a cloud host.

Week 23 — MLOps

The ML lifecycle end to end: experiment tracking (MLflow), model registry, data/model versioning, CI/CD for ML, monitoring for model drift and data drift, retraining pipelines, structured logging, and production architecture patterns.

Week 24 — Interview Preparation

Dedicate this week to Parts 5, 6 and 11 of this book: the question banks, system-design case studies and cheat sheets. Do at least two mock interviews.

MILESTONE CHECK

A deployed, monitored AI service; a polished resume, GitHub and LinkedIn; and daily job applications running by Week 24 (Part 10).

The 180-Day Study Plan

24 weeks, 2–4 hours a day. Each week gives a daily rhythm — six focused study days and one lighter review/catch-up day — plus the deliverable that proves the week actually landed. Checkpoints every 30 days tell you exactly what to self-test.

PRO TIP

Block the same 2–4 hour window every day. Consistency beats intensity — six focused 2-hour days beat one exhausted 12-hour weekend.

Month 1 — Weeks 1–4 · Days 1–30

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
1	Variables → loops → functions → strings/lists → dicts/exceptions	Timed coding challenge (10 problems)	Review + rewrite weak solutions	20 solved Python problems
2	OOP → modules/files → JSON → Git/GitHub → venv/pip	Build the API-consumer mini project	Polish README, push to GitHub	API mini project live on GitHub
3	NumPy arrays/broadcasting → Pandas cleaning → groupby/merge	Clean a real messy dataset	Write cleaning-steps summary	Cleaned dataset + notebook
4	Linear algebra intuition → probability/stats → gradient descent	Start Month 1 EDA project	Finish EDA + Day 30 checkpoint	Full EDA project + report

✓ CHECKPOINT — DAY 30

- Write a Python script using functions, loops and error handling from scratch
- Clean a messy CSV with Pandas (missing values, duplicates, type fixes)
- Explain gradient descent in one sentence to a non-technical friend
- Push a project to GitHub with a working README

Month 2 — Weeks 5–8 · Days 31–60

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
5	Supervised/unsupervised/RL overview → splits/leakage → bias-variance	10 ML concept quiz questions	Review weak areas	ML fundamentals notes doc
6	Linear/poly regression → Ridge/Lasso → MAE/RMSE/R ²	Build House Price Predictor	Tune + document results	House Price Predictor project
7	Logistic/KNN/trees/forests/SVM/XGBoost → precision/recall/F1	Build Churn Predictor	Full metrics report	Churn Predictor project
8	K-Means/PCA → encoding/scaling → imbalance → GridSearchCV	Assemble end-to-end ML pipeline	Finish pipeline + Day 60 checkpoint	Production-style ML project

✓ CHECKPOINT — DAY 60

- Build, tune and evaluate a classifier and a regressor from a raw dataset
- Explain precision vs. recall with a real example (e.g. spam vs. fraud detection)
- Justify a metric choice for an imbalanced dataset
- Have 2 polished ML projects on GitHub with READMEs

Month 3 — Weeks 9–12 · Days 61–90

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
9	Neurons/layers/activations → loss functions → forward/back-prop	Implement a tiny NN by hand (no framework)	Review + quiz	From-scratch neural net notebook
10	Tensors → Datasets/DataLoaders → training loop → GPU → save/load	Train an MNIST-style classifier in PyTorch	Refactor training loop cleanly	PyTorch training-loop template
11	Convolution/pooling → augmentation → transfer learning	Build image classifier	Deploy simple demo UI	Image Classifier project
12	Tokenization/TF-IDF/embeddings → attention intro	Build sentiment classifier	Finish + Day 90 checkpoint	Sentiment Classifier project

✓ CHECKPOINT — DAY 90 (HALFWAY)

- Write and train a PyTorch model from a blank file, unaided
- Explain what a convolution layer does and why CNNs suit images
- Have 4–5 solid projects on GitHub spanning ML and DL
- Ready to start Generative AI — the highest-leverage months ahead

Month 4 — Weeks 13–16 · Days 91–120

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
13	Self-attention → multi-head attention → positional encoding → encoder/decoder	Trace attention on a toy sentence by hand	Diagram the full transformer block	Annotated transformer diagram
14	Pretraining/fine-tuning → temperature/top-p → hallucinations → tool calling	Experiment with prompt patterns via an API	Write a prompt-design cheat sheet	Personal prompt-pattern library
15	Streaming → structured output → tool calling → cost/security	Build the AI Assistant project	Add error handling + rate-limit logic	AI Assistant Application
16	Chunking → embeddings → vector DB → retrieval → reranking	Build Chat-With-PDFs assistant	Finish + Day 120 checkpoint	RAG Knowledge-Base Assistant

✓ CHECKPOINT — DAY 120

- Explain self-attention and why transformers replaced RNNs, in plain language
- Build a working RAG pipeline over your own documents from scratch
- Have a live, demoable GenAI project — your strongest interview talking point

Month 5 — Weeks 17–20 · Days 121–150

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
17	Query rewriting → hybrid retrieval → multi-query → context compression	Upgrade Month 4's RAG with reranking	Run a RAG evaluation pass	Upgraded, evaluated RAG system
18	Agent loops → tools/memory → multi-agent → human-in-the-loop	Build a single-tool agent	Extend to the Research Agent project	Research Agent project
19	Fine-tuning vs. RAG → LoRA/QLoRA/PEFT → quantization	Read 2 fine-tuning case studies	Summarize when you'd fine-tune	Fine-tuning decision framework note
20	Groundedness/relevance → prompt injection → guardrails → observability	Add evaluation + logging to your agent	Finish + Day 150 checkpoint	Production-style AI Agent / RAG app

✓ CHECKPOINT — DAY 150

- Ship an agent or advanced-RAG project you can demo live in an interview
- Explain your evaluation approach — how you know your AI system actually works
- Portfolio of 6–7 projects; ready to start deployment & job search

Month 6 — Weeks 21–24 · Days 151–180

WEEK	DAYS 1–5 FOCUS	DAY 6	DAY 7	DELIVERABLE
21	FastAPI → Pydantic → async → model serving → auth/logging	Wrap a model in a FastAPI service	Add validation + error handling	FastAPI model-serving project
22	Dockerfile/images → Compose → secrets → cloud/GPU overview	Containerize your FastAPI service	Deploy it to a cloud host	Live, deployed AI application
23	MLflow → registry → versioning → CI/CD → drift monitoring	Add experiment tracking to a project	Write a production -architecture doc	MLOps-instrumented project
24	Work Parts 5, 6 & 11 daily — full-stack review	Mock interview #1	Mock interview #2 + Day 180 checkpoint	Interview-ready across all tracks

✓ CHECKPOINT — DAY 180 (FINAL)

- A deployed AI application with monitoring — a link you can hand a recruiter
- 8+ portfolio-quality projects, resume, GitHub and LinkedIn all aligned
- Comfortable in mock interviews across ML, DL, GenAI and system design
- Applying daily using the Part 10 job-search plan

15 Premium Portfolio Projects

Three flagship projects — one per difficulty tier — get the full treatment: problem, architecture, folder structure, deployment and resume bullets. The remaining twelve are summarized in the table that follows; apply the same treatment to any of them using a flagship as a template.

Flagship 1 (Beginner) — Sentiment Analyzer

FIELD	DETAIL
Problem	Classify product/movie reviews as positive, negative or neutral.
Features	Text input, batch CSV upload, confidence score, simple web demo.
Tech stack	Python, scikit-learn/PyTorch, TF-IDF or embeddings, Streamlit/FastAPI.
Architecture	Raw text → preprocessing → vectorizer → classifier → confidence-scored label → UI.
Dataset	IMDB reviews or Amazon reviews (Kaggle).
Steps	1) Clean text 2) Vectorize (TF-IDF) 3) Train & compare 2–3 classifiers 4) Pick best by F1 5) Wrap in a UI 6) Deploy.
Folder structure	src/ (preprocess.py, train.py, predict.py) · app/ (streamlit_app.py) · data/ · models/ · README.md
Deployment	Streamlit Community Cloud or a small Docker container on a free-tier host.
Resume bullet	"Built and deployed a sentiment-analysis web app achieving 89% F1 on 25k reviews, comparing 3 classical models."
Interview Qs	Why TF-IDF over embeddings here? How would you handle sarcasm/negation?
Make it unique	Add aspect-based sentiment (sentiment per product feature) or a confidence-calibration chart.

Flagship 2 (Intermediate) — PDF Q&A Chatbot

FIELD	DETAIL
Problem	Let a user upload PDFs and ask natural-language questions, answered with citations.
Features	Multi-PDF upload, chunked ingestion, streamed answers, source citations, chat history.
Tech stack	Python, LangChain or LlamaIndex, an LLM API, Chroma/FAISS, FastAPI + a simple frontend.
Architecture	PDF → text extraction → chunking → embedding → vector store → (query) retrieve top-k → construct prompt with citations → LLM → streamed answer.
Dataset	Your own PDFs — resumes, textbooks, company handbooks.
Steps	1) Extract & chunk text 2) Embed & store in vector DB 3) Build retrieval + prompt template 4) Add streaming 5) Add citation formatting 6) Deploy behind FastAPI.
Folder structure	ingestion/ · retrieval/ · api/ (main.py) · frontend/ · vectorstore/ · tests/ · README.md
Deployment	Dockerized FastAPI on Render/Railway/AWS; vector store can stay local (FAISS) or managed (Pinecone).
Resume bullet	"Engineered a RAG-based PDF Q&A; assistant with source citations, cutting document lookup time by ~70% in testing."

FIELD	DETAIL
Interview Qs	How do you choose chunk size? How do you stop it answering from outside the PDFs?
Make it unique	Add hybrid search (keyword + vector) or a "confidence: low — verify manually" flag when retrieval scores are weak.

Flagship 3 (Advanced) — Production RAG Assistant

FIELD	DETAIL
Problem	A RAG assistant robust enough to run in front of real users — evaluated, monitored, cost-aware.
Features	Hybrid retrieval, reranking, caching, evaluation dashboard, drift/cost monitoring, guardrails.
Tech stack	FastAPI, LangChain/LlamaIndex, Pinecone/Chroma, a reranker model, Redis (caching), MLflow or a tracing tool, Docker.
Architecture	Ingestion pipeline → hybrid retrieval (BM25 + vector) → rerank → cache layer → LLM generation → response + citations → async logging to an evaluation/observability store.
Dataset	A realistic knowledge base — public docs (e.g. a product's help center) or your own corpus.
Steps	1) Build hybrid retrieval 2) Add reranking 3) Add response caching 4) Build an eval set + score groundedness/relevance 5) Add cost/latency logging 6) Containerize & deploy 7) Add a monitoring dashboard.
Folder structure	ingestion/ · retrieval/ (hybrid.py, rerank.py) · api/ · eval/ (eval_set.json, run_eval.py) · monitoring/ · infra/ (Dockerfile, compose.yml) · README.md
Deployment	Docker Compose (API + cache + vector DB) on a cloud VM or managed container service.
Resume bullet	"Built a production-grade RAG system with hybrid retrieval, reranking and caching, reducing p95 latency by 40% and hallucination rate by 30% via an automated eval suite."
Interview Qs	How do you evaluate a RAG system without human labels for every query? How do you keep costs bounded at scale?
Make it unique	Add an automated regression test that fails CI if groundedness drops below a threshold on your eval set.

All 15 Projects at a Glance

Apply the flagship template (problem → features → stack → architecture → steps → folders → README → deployment → resume bullet → interview Qs → uniqueness) to any project below.

#	PROJECT	CORE STACK	KEY FEATURE TO BUILD
B1	House Price Predictor	scikit-learn, Pandas	Regularized regression + CV
B2	Spam Classifier	scikit-learn, NLTK	Naive Bayes vs. gradient boosting comparison
B3	Customer Churn Predictor	XGBoost, SHAP	Feature importance explanation
B4	Sentiment Analyzer	TF-IDF/embeddings, Streamlit	See Flagship 1
B5	Image Classifier	PyTorch, transfer learning	Fine-tune a pretrained CNN backbone
I1	Resume Analyzer	LLM API, structured output	JSON-schema resume scoring against a JD
I2	AI Interview Prep Assistant	LLM API, prompt chaining	Role-specific Q generation + answer feedback
I3	PDF Q&A; Chatbot	RAG stack	See Flagship 2
I4	AI Customer Support Assistant	RAG + tool calling	Escalation-to-human handoff logic
I5	Recommendation System	Collaborative filtering / embeddings	Cold-start handling strategy

#	PROJECT	CORE STACK	KEY FEATURE TO BUILD
A1	Production RAG Assistant	Hybrid retrieval, reranking, eval	See Flagship 3
A2	AI Research Agent	Agent framework, search tools	Multi-step plan-then-execute loop
A3	Multi-Agent Application	Agent framework, message bus	Specialized agents + a coordinator
A4	AI Resume Screening Platform	LLM + embeddings, FastAPI	Bias-aware ranking + explanations
A5	Full-Stack AI SaaS Application	FastAPI, Docker, auth, billing stub	End-to-end: auth → AI feature → deployment → docs

Coding & Interview Question Banks

A curated, high-value set per topic with real answers — representative of the 500+ practice problems and 200+ interview questions worth drilling; use these as your revision core and generate more of the same shape as you practice.

Python

Q1 (Beg). Difference between a list and a tuple?

A: Lists are mutable, tuples immutable; tuples are hashable and slightly faster, used for fixed collections.

Q2 (Beg). What is a Python generator?

A: A function using `yield` that produces values lazily, one at a time, saving memory over building a full list.

Q3 (Beg). `is` vs `==`?

A: `==` compares values; `is` compares object identity (same memory reference).

Q4 (Int). What are `*args` and `**kwargs`?

A: Ways to accept variable positional and keyword arguments respectively in a function signature.

Q5 (Int). Explain Python's GIL.

A: The Global Interpreter Lock allows only one thread to execute Python bytecode at a time — CPU-bound work needs multiprocessing, not threading, for true parallelism.

Q6 (Int). Difference between `@staticmethod` and `@classmethod`?

A: A static method takes no implicit first arg; a class method takes `cls` and can access/modify class state.

Q7 (Adv). How do decorators work?

A: A decorator is a function that wraps another function, extending behavior without modifying its source — implemented via closures.

Q8 (Adv). Shallow vs. deep copy?

A: Shallow copy duplicates the outer object but shares nested references; deep copy recursively duplicates everything.

NumPy & Pandas

Q1. What is broadcasting in NumPy?

A: NumPy's rule for applying operations across arrays of different but compatible shapes without explicit loops.

Q2. `loc` vs `iloc` in Pandas?

A: `loc` selects by label, `iloc` by integer position.

Q3. How do you handle missing values?

A: `dropna()` to remove, `fillna()` with mean/median/mode or a model-based imputation, depending on the missingness pattern.

Q4. What does `groupby().agg()` do?

A: Splits data into groups by key, applies one or more aggregation functions per group, and combines the results.

Q5. Vectorized ops vs. `.apply()` — which is faster and why?

A: Vectorized ops run in compiled C loops; `.apply()` runs a Python function per row/column and is much slower at scale.

Machine Learning

Q1 (Beg). What is overfitting and how do you detect it?

A: The model memorizes training data and fails to generalize; detect via a large gap between train and validation performance.

Q2 (Beg). Precision vs. recall?

A: Precision = correct positive predictions ÷ all positive predictions; recall = correct positive predictions ÷ all actual positives.

Q3 (Int). How does regularization (L1/L2) help?

A: It penalizes large weights, reducing overfitting; L1 (Lasso) can zero out features, L2 (Ridge) shrinks them smoothly.

Q4 (Int). How does Random Forest reduce variance vs. a single tree?

A: It averages many decorrelated trees trained on bootstrapped samples and random feature subsets.

Q5 (Int). What is cross-validation for?

A: To get a more reliable estimate of model performance by training/testing across multiple folds instead of one split.

Q6 (Adv). Explain gradient boosting vs. bagging.

A: Bagging trains models in parallel on random subsets and averages them (reduces variance); boosting trains models sequentially, each correcting the previous one's errors (reduces bias).

Q7 (Adv). How would you handle severe class imbalance?

A: Resampling (SMOTE/undersampling), class-weighted loss, and evaluating with F1/PR-AUC instead of accuracy.

Deep Learning

Q1 (Beg). Why do we need activation functions?

A: Without them, stacked layers collapse into one linear function — activations introduce non-linearity so networks can learn complex patterns.

Q2 (Beg). What is backpropagation?

A: The algorithm that computes the gradient of the loss with respect to every weight via the chain rule, used to update weights.

Q3 (Int). Vanishing gradient problem — cause and fix?

A: Deep networks with saturating activations (sigmoid/tanh) shrink gradients to near-zero in early layers; fixed with ReLU, residual connections, and normalization.

Q4 (Int). Why does a CNN use fewer parameters than a fully-connected net on images?

A: Convolution kernels are shared across spatial locations instead of learning a unique weight per pixel connection.

Q5 (Adv). Batch normalization — what does it actually do?

A: Normalizes layer activations per batch, stabilizing and speeding up training and reducing sensitivity to initialization.

Generative AI & LLMs

Q1 (Beg). What is a token?

A: A chunk of text (often a sub-word) that an LLM processes as one unit — not always a full word.

Q2 (Beg). What does "context window" mean?

A: The maximum number of tokens (input + output) a model can consider at once in a single call.

Q3 (Beg). What does temperature control?

A: Randomness in generation — near 0 is deterministic/focused, higher values increase diversity and creativity.

Q4 (Int). What is a hallucination and why does it happen?

A: A confident but factually wrong or fabricated output; happens because the model predicts plausible-sounding text, not verified facts.

Q5 (Int). Prompt engineering vs. fine-tuning — when to use which?

A: Prompting for most behavior/format changes (fast, cheap, flexible); fine-tuning when you need a consistent style/domain behavior prompting can't reliably achieve at scale.

Q6 (Int). What is function/tool calling?

A: A structured way for a model to request a specific function be invoked with specific arguments, so it can take real actions or fetch real data.

Q7 (Adv). How do you get reliable structured (JSON) output from an LLM?

A: Use the API's schema/structured-output mode when available, validate with a schema library (e.g. Pydantic), and retry on validation failure.

Q8 (Adv). How would you reduce inference cost in a high-traffic LLM feature?

A: Cache repeated queries, route simple requests to a smaller/cheaper model, shorten prompts, and batch where possible.

RAG

Q1 (Beg). Why use RAG instead of just a bigger model?

A: RAG grounds answers in your actual, current data without retraining — cheaper, faster to update, and more auditable.

Q2 (Beg). What is chunking and why does size matter?

A: Splitting documents into retrievable pieces; too large loses precision, too small loses context — tune per document type.

Q3 (Int). What is reranking and when do you need it?

A: A second, more precise scoring pass over initial retrieval results, useful when the first-pass retriever is fast but imprecise.

Q4 (Int). What is hybrid search?

A: Combining keyword (BM25) and vector similarity search, catching both exact-term matches and semantic matches.

Q5 (Adv). How do you evaluate a RAG system?

A: Score groundedness (is the answer supported by retrieved context?), answer relevance, and retrieval precision/recall on a labeled eval set.

Q6 (Adv). How do you handle a query the knowledge base can't answer?

A: Detect low retrieval-similarity scores and have the system explicitly say it doesn't know, rather than guessing.

AI Agents

Q1 (Beg). What makes an "agent" different from a single LLM call?

A: An agent runs a loop — reason, act via a tool, observe the result, repeat — rather than answering in one shot.

Q2 (Int). Why is agent reliability harder than a normal API?

A: Errors compound across steps and agents can fail silently in creative ways — one bad tool call can derail the whole chain.

Q3 (Int). What is "memory" in an agent system?

A: Short-term (conversation context) and long-term (a store the agent can read/write across sessions, often a vector DB).

Q4 (Adv). When would you use a multi-agent system over a single agent?

A: When a task naturally splits into specialized roles (e.g. researcher + writer + reviewer) that benefit from separate prompts/context.

Q5 (Adv). How do you keep an agent safe for consequential actions?

A: Add human-in-the-loop approval steps, restrict tool permissions, and log every action for auditability.

PRO TIP

Don't just memorize these answers — rebuild each concept out loud from scratch once a week. Interviewers probe follow-ups; fluency beats memorization every time.

AI System Design

"Design an AI-powered application" is now a standard senior-track question and increasingly asked of freshers too. Answer it by walking through the same layers every time.

The framework — walk through every layer

LAYER	WHAT TO COVER
Requirements	Who uses it, what "correct" means, latency/cost/accuracy targets.
Data pipeline	Sources, ingestion frequency, cleaning, storage.
Model selection	Off-the-shelf LLM API vs. fine-tuned vs. classical model — justify the trade-off.
Embeddings & retrieval	Vector DB choice, chunking strategy, hybrid search where relevant.
LLM & API layer	Prompt design, streaming, structured outputs, tool calling.
Caching & DBs	What's cacheable, what needs a transactional store vs. a vector store.
Scaling & latency	Async processing, queueing, horizontal scaling of the API layer.
Cost	Token budgets, model tiering, caching to cut repeat calls.
Security	Prompt-injection defenses, data access controls, PII handling.
Monitoring & eval	Logging prompts/responses, drift detection, an ongoing eval set.

INTERVIEW TIP

Start every system-design answer by asking 2–3 clarifying questions about scale and accuracy needs. Interviewers score how you scope a fuzzy problem as much as the final architecture.

Case studies — sketch the architecture for each

- **ChatGPT-like assistant** — chat UI → conversation store → LLM API with streaming → optional tool calling → moderation layer → usage/cost logging.
- **Enterprise document Q&A** — document ingestion pipeline → chunk/embed → vector DB with access-control metadata → retrieval scoped to the user's permissions → LLM answer with citations.
- **AI customer support** — ticket intake → RAG over help docs → confidence-based routing (auto-resolve vs. escalate to human) → feedback loop to improve the knowledge base.
- **AI resume screener** — resume parsing → structured extraction (skills, experience) → embedding-based match against JD → explainable ranking → bias-review checkpoint before any auto-rejection.
- **Recommendation engine** — user/item embeddings → candidate generation (approximate nearest neighbor) → re-ranking model with business rules → A/B-tested rollout.
- **AI coding assistant** — code context retrieval from the repo → LLM completion/edit suggestion → static-analysis/test-run validation before applying → telemetry on acceptance rate.

- **Content moderation system** — classifier/LLM screening pipeline → confidence thresholds → human review queue for edge cases → continuous relabeling of hard examples.
- **Voice assistant** — speech-to-text → intent/LLM reasoning layer → tool calling for actions → text-to-speech → latency budget across the whole chain.
- **Fraud detection system** — real-time feature pipeline → low-latency classical ML model (not an LLM — latency and cost matter) → human review for flagged cases → feedback loop for retraining.
- **Personalized learning platform** — student progress store → knowledge-gap model → RAG over course content → LLM-generated practice questions tuned to the gap.

GitHub & Portfolio Strategy

What an AI Engineer's GitHub should look like

3–5 pinned, polished repos beat 30 half-finished ones. Each pinned repo needs: a clear name, a real README, a working demo link, and commits that show iteration — not one giant initial commit.

- **Naming:** descriptive and lowercase-hyphenated (rag-pdf-assistant, not project1)
- **README:** problem → demo GIF/link → architecture diagram → setup → results → limitations
- **Architecture diagrams:** a simple boxes-and-arrows image beats a wall of text — even a hand-drawn one
- **Demo videos:** a 30–60s screen recording embedded or linked in the README
- **Commit strategy:** small, meaningful commits with clear messages — this is read as a proxy for how you work
- **Documentation:** docstrings on public functions, a requirements file, a one-command setup
- **Deployment links:** a live URL wherever feasible — recruiters click, they rarely clone

Portfolio website structure

One page is enough: a one-line positioning statement, 3–5 project cards (image, one-line pitch, live link, GitHub link), a short about section, and contact/resume links. Avoid a generic template — swap in your own project screenshots.

JOB-READY PORTFOLIO CHECKLIST

- ☐ 3–5 pinned, polished repos
- ☐ Each has a demo link or video
- ☐ Each has an architecture diagram
- ☐ READMEs follow one consistent structure
- ☐ At least 1 deployed, live AI application
- ☐ Portfolio site with all projects linked
- ☐ GitHub profile README written
- ☐ Commit history shows real iteration

The AI Engineer Resume

One page. ATS-friendly means plain formatting, standard section headers, and no tables/graphics inside the resume itself (the box below is a content template, not a layout to copy literally).

Dipanshu

Aspiring AI Engineer · dipanshu.decoded · LinkedIn · GitHub · Portfolio

PROFESSIONAL SUMMARY

Aspiring AI Engineer with hands-on experience building ML, deep learning and generative-AI applications, including a deployed production-style RAG system. Skilled in Python, PyTorch and LLM application development.

SKILLS

Python, SQL, NumPy/Pandas, scikit-learn, PyTorch, LangChain/LlamaIndex, FastAPI, Docker, MLflow, AWS/GCP basics

PROJECTS

- Production RAG Assistant — hybrid retrieval + reranking, cut hallucination rate 30% via automated eval [GitHub / Live Demo]
- AI Research Agent — multi-step planning agent producing structured reports from live web sources [GitHub / Live Demo]

EDUCATION

B.Tech / B.Sc. in [Field], [University], [Year]

CERTIFICATIONS

[Relevant course/certification, if any]

20+ resume bullet patterns

- "Built and deployed a [project] achieving [metric] on [dataset/scale], using [tools]."
- "Engineered a RAG pipeline over [N] documents, reducing manual lookup time by [X]%."
- "Fine-tuned/prompt-engineered [model] to improve [task metric] from [X] to [Y]."
- "Designed an evaluation suite that reduced hallucination rate by [X]% before deployment."
- "Containerized and deployed [service] on [cloud], serving [X] requests/day at [latency]."
- "Built a multi-agent system automating [task], cutting manual effort by [X] hours/week."
- "Implemented hybrid search (BM25 + vector) improving retrieval precision by [X]%."
- "Instrumented an ML pipeline with MLflow for experiment tracking across [N] model versions."

Pattern: action verb + what you built + technique used + measured result. Always fill in a real (even approximate) number — "improved accuracy" is forgettable, "improved F1 from 0.71 to 0.84" is not.

LinkedIn & Personal Branding

Headline examples

- "Aspiring AI Engineer | Building with LLMs, RAG & Agents | Python · PyTorch · FastAPI"
- "AI/ML Engineer in the making | Turning models into shipped products"
- "Generative AI Engineer (Fresher) | RAG Systems · AI Agents · Production ML"

About section template

I'm an aspiring AI Engineer who spent the last 6 months going from Python fundamentals to building production-style RAG systems and AI agents. I care about shipping AI that actually works in production — not just in a notebook. Currently building: [current project]. Open to AI Engineer / GenAI Engineer roles. Let's connect if you're working on applied AI. — dipanshu.decoded

Featured section & project-post template

Pin your top 2–3 projects (with demo links) to Featured. For a project post: hook (the problem) → what you built (1–2 lines) → one interesting technical decision → demo link/GIF → what you'd improve next. End with a question to invite comments.

Networking, outreach & referral templates

Recruiter outreach: "Hi [Name], I noticed [Company] is hiring for [role]. I've spent the last 6 months building production AI systems — most recently a RAG assistant that [one metric]. Would love to connect and learn more about the team."

Referral request: "Hi [Name], I saw you work on [team] at [Company] — I'm applying for [role] and would really value your perspective. Would you be open to a quick chat, or a referral if it feels like a fit after we talk?"

30-day LinkedIn posting plan

WEEK	POST THEMES (2–3 POSTS)
1	Introduce your roadmap; share one "thing I learned this week" post
2	Deep-dive post on one technical concept (e.g. "how RAG actually works")
3	Launch post for a finished project, with demo link
4	Reflection post + comment/engage on 5 other people's posts daily

The 30-Day Job Search Plan

Where to find genuine entry-level AI roles

Company careers pages, LinkedIn Jobs (filter "Entry level"), AngelList/Wellfound for startups, and niche AI job boards. A "genuine" entry-level posting rarely demands 3+ years of production LLM experience — treat those as senior postings mislabeled, and don't let them dent your confidence.

Startup vs. MNC strategy

Startups: faster interview loops, more willingness to bet on strong projects over pedigree, broader scope.

MNCs/services companies: more structured hiring, often better for a first stable role and process-based interview prep. Apply to both in parallel.

WEEK	DAILY TARGET	FOCUS
1	8–10 applications	Resume/LinkedIn finalized; start a tracker; 5 cold outreach messages/day
2	8–10 applications	Referral requests to 2nd-degree connections; publish 1 project post
3	6–8 applications	Prioritize interview prep for any responses; keep outreach running
4	6–8 applications	Mock interviews; follow up on all pending applications; refine based on feedback

PRO TIP

Track every application in one sheet: company, role, date, status, contact. A referral is worth roughly 5–10x a cold application — spend real time on outreach, not just volume-applying.

Interview Cheat Sheets

One glance, one revision. Read these the night before an interview.

Python

Mutability, OOP (inheritance/polymorphism), decorators, generators, GIL, list/dict comprehensions, exception handling.

SQL

JOINS, GROUP BY + HAVING, window functions, indexing basics, normalization, subqueries vs. CTEs.

Statistics

Mean/median/mode, variance/std dev, distributions, p-values, hypothesis testing, correlation $\neq$ causation.

Machine Learning

Bias-variance, overfitting, regularization, cross-validation, ensemble methods, evaluation metrics per problem type.

Deep Learning

Forward/backprop, activation functions, vanishing/exploding gradients, batch norm, dropout, optimizers (SGD/Adam).

NLP

Tokenization, TF-IDF, embeddings, Word2Vec intuition, attention, sequence-to-sequence basics.

Transformers

Self/multi-head attention, positional encoding, encoder vs. decoder, why they parallelize better than RNNs.

LLMs

Pretraining vs. fine-tuning, temperature/top-p, hallucinations, context window limits, tool calling.

Prompt Engineering

System vs. user prompts, few-shot examples, chain-of-thought, structured/JSON output, iterative refinement.

RAG

Chunking, embeddings, retrieval, reranking, hybrid search, groundedness evaluation.

Vector Databases

Approximate nearest neighbor search, cosine vs. dot-product similarity, indexing (HNSW), metadata filtering.

AI Agents

Agent loops, tool calling, memory, planning, multi-agent coordination, reliability/guardrails.

FastAPI

Path/query/body params, Pydantic validation, async endpoints, dependency injection, error handling.

Docker

Image vs. container, Dockerfile layers, Compose for multi-service apps, env vars vs. secrets.

MLOps

Experiment tracking, model registry, CI/CD for models, data/model drift, retraining triggers.

AI System Design

Requirements → data → model → retrieval → serving → caching → scaling → cost → security → monitoring.

Final Job-Ready Checklist & Your Journey

Programming

- ☐ Python
- ☐ SQL
- ☐ Git/GitHub

Machine Learning

- ☐ ML algorithms
- ☐ Feature engineering
- ☐ Model evaluation

Deep Learning

- ☐ Neural networks
- ☐ PyTorch
- ☐ CNNs
- ☐ Transformers

Generative AI

- ☐ LLMs
- ☐ Prompt engineering
- ☐ Embeddings
- ☐ RAG
- ☐ Vector databases
- ☐ Agents
- ☐ Evaluation

Production

- ☐ FastAPI
- ☐ Docker
- ☐ Cloud
- ☐ MLOps
- ☐ Monitoring

Career

- ☐ 5+ strong projects
- ☐ GitHub optimized

- ☐ Resume optimized
- ☐ LinkedIn optimized
- ☐ Interview prep completed

SUMMARY CHECKLIST

- ☐ I can build and evaluate an ML model end-to-end without a tutorial open beside me
- ☐ I can write a PyTorch training loop from memory
- ☐ I can explain transformer attention in plain language
- ☐ I've built one real RAG pipeline outside of a walkthrough
- ☐ I've deployed at least one model behind an API and understand basic monitoring
- ☐ I have 3-5 deployed, documented portfolio projects
- ☐ I can answer project deep-dive questions for 5 minutes without notes
- ☐ My resume and GitHub would make sense to a stranger in under 60 seconds

Final note: Six months is realistic only if each box above is actually true, not just covered. Each module is the foundation the next one leans on — skipping the milestone check to move faster is how people end up with six months of notes and no offers.

CUT SCOPE, NOT DEPTH

Ship fewer projects done well rather than rushing all 15+ shallowly. Two excellent projects out-interview five sloppy ones.

EXTEND, DON'T SKIP

A 7-8 month timeline with every milestone genuinely passed beats a 6-month timeline with three of them faked.

CLOSING

Your AI Engineer Journey

Beginner	Python	Data	Machine Learning	Deep Learning
Transformers	LLMs	RAG	AI Agents	FastAPI
MLOps	Deployment	Portfolio	Interviews	AI Engineer

“You don't need to learn everything in AI. You need to learn the right things, build real projects, and prove that you can turn AI models into useful products.”

— dipanshu.decoded

AI ENGINEER ROADMAP · dipanshu.decoded · 2026 EDITION